

DGM: A deep learning algorithm for solving partial differential equations

Justin Sirignano* and Konstantinos Spiliopoulos^{†‡§}

November 14, 2021

Abstract

High-dimensional PDEs have been a longstanding computational challenge. We propose to solve high-dimensional PDEs by approximating the solution with a deep neural network which is trained to satisfy the differential operator, initial condition, and boundary conditions. We prove that the neural network converges to the solution of the partial differential equation as the number of hidden units increases. Our algorithm is meshfree, which is key since meshes become infeasible in higher dimensions. Instead of forming a mesh, the neural network is trained on batches of randomly sampled time and space points. We implement the approach for American options (a type of free-boundary PDE which is widely used in finance) in up to 200 dimensions. We call the algorithm a “Deep Galerkin Method (DGM)” since it is similar in spirit to Galerkin methods, with the solution approximated by a neural network instead of a linear combination of basis functions.

1 Deep learning and high-dimensional PDEs

High-dimensional partial differential equations (PDEs) have been a longstanding computational challenge. Finite difference methods become infeasible in higher dimensions due to the explosion in the number of grid points and sometimes the demand for reduced time step size. If there are d space dimensions and 1 time dimension, the mesh is of size $\mathcal{O}^{d+1}$. Galerkin methods are a widely-used computational method which seeks a reduced-form solution to a PDE as a linear combination of basis functions. We propose a deep learning algorithm which is similar in spirit to the Galerkin method, but with several key changes using ideas from machine learning. The deep learning algorithm, or “Deep Galerkin Method” (DGM), uses a deep neural network instead of a linear combination of basis functions. The deep neural network is trained to satisfy the differential operator, initial condition, and boundary conditions using stochastic gradient descent at randomly sampled spatial points. By randomly sampling spatial points, we avoid the need to form a mesh (which is infeasible in higher dimensions) and instead convert the PDE problem into a machine learning problem.

DGM is a natural merger of Galerkin methods and machine learning. The algorithm in principle is straightforward; see Section 2. Promising numerical results are presented later in Section 4 for American options (a free-boundary PDE that is widely used in finance). DGM converts the computational cost of finite difference to a more convenient form: instead of a huge mesh of $\mathcal{O}^{d+1}$ (which is infeasible to handle), many batches of random spatial points are generated. Although the total number of spatial points could be vast, the algorithm can process the spatial points sequentially without harming the convergence rate. The algorithm can in principle be applied to nearly any PDE (hyperbolic, elliptic, parabolic, and partial-integral differential equations).

*University of Illinois at Urbana Champaign, Urbana, E-mail: jasirign@illinois.edu

[†]Department of Mathematics and Statistics, Boston University, Boston, E-mail: kspiliop@math.bu.edu

[‡]The authors thank seminar participants at the JP Morgan Machine Learning and AI Forum seminar, the Imperial College London Applied Mathematics and Mathematical Physics seminar, Princeton University, and Northwestern University for their comments. The authors would also like to thank participants at the 2017 INFORMS Applied Probability Conference and participants at the 2017 Greek Stochastics Conference for comments.

[§]Research of K.S. supported in part by the National Science Foundation (DMS 1550918). Computations for this paper were performed using the Blue Waters supercomputer grant “Distributed Learning with Neural Networks”.

We also prove a theorem regarding the approximation power of neural networks for quasilinear partial differential equations. Consider the potentially nonlinear PDE

$$\begin{aligned}\partial_t u(t, x) + \mathcal{L}u(t, x) &= 0, & (t, x) &\in [0, T] \times \Omega \\ u(0, x) &= u_0(x), & x &\in \Omega \\ u(t, x) &= g(t, x), & x &\in [0, T] \times \partial\Omega.\end{aligned}\tag{1.1}$$

The solution $u(t, x)$ is of course unknown, but an approximate solution $f(t, x)$ can be found by minimizing the L^2 error

$$J(f) = \|\partial_t f + \mathcal{L}f\|_{2, [0, T] \times \Omega}^2 + \|f - g\|_{2, [0, T] \times \partial\Omega}^2 + \|f(0, \cdot) - u_0\|_{2, \Omega}^2.$$

The error function $J(f)$ measures how well the approximate solution f satisfies the differential operator, boundary condition, and initial condition. Note that no knowledge of the actual solution u is assumed; $J(f)$ can be directly calculated from the PDE (1.1) for any approximation f . If $J(f) = 0$, then f is a solution to the PDE. Define $\mathfrak{C}^n$ as the class of neural networks with n hidden units and let f_n be a neural network with n hidden units which minimizes $J(f)$. We prove that

$$\begin{aligned}f_n &= \arg \min_{f \in \mathfrak{C}^n} J(f), \\ f_n &\rightarrow u \quad \text{as } n \rightarrow \infty,\end{aligned}$$

strongly in, at least, $L^1([0, T] \times \Omega)$ for quasilinear PDEs; see Theorem 5.4 for the precise statement. That is, the neural network will converge to the solution of the PDE as the size of the network grows large. The precise statement of the theorem and its proof are presented in Section 5. The proof requires the joint analysis of the approximation power of neural networks as well as the continuity properties of partial differential equations. Note that $J(f_n) \rightarrow 0$ does not necessarily imply that $f_n \rightarrow u$, given that we only have L^2 control on the approximation error. First, we prove that $J(f_n) \rightarrow 0$ as $n \rightarrow \infty$. We then establish that each neural network $\{f_n\}_{n=1}^\infty$ satisfies a PDE with a source term $h^n(t, x)$. Importantly, we only know that the source terms $h^n(t, x)$ are bounded in L^2 norm. We are then able to prove the convergence of $f_n \rightarrow u$ as $n \rightarrow \infty$ in $L^1([0, T] \times \Omega)$ and in $L^\rho(0, T; W^{1, \rho}(\Omega))$ for every $\rho < 2$ using compactness arguments.

The numerical analysis in this paper focuses primarily on solving American option PDEs in high dimensions. An American option is a financial derivative on a portfolio of stocks. Financial institutions are interested in pricing options on portfolios ranging from dozens to even hundreds of stocks [16]. The price of the American option satisfies a free-boundary partial differential equation where the number of space dimensions equals the number of stocks in the portfolio. The PDE solution also provides the optimal hedging policy to reduce and manage risk. The American option PDE is widely used by financial institutions and banks across the world. There is a significant need for numerical methods to accurately solve high-dimensional American option PDEs.

Evaluating the accuracy of the deep learning algorithm is not straightforward. PDEs with semi-analytic solutions may not be sufficiently challenging. (After all, the semi-analytic solution exists since the PDE can be transformed into a lower-dimensional equation.) It cannot be benchmarked against traditional finite difference (which fails in high dimensions). The American option PDE provides a unique opportunity to test the deep learning algorithm. It has the special property that error bounds can be calculated for any approximate solution. This allows for us to evaluate the accuracy of the deep learning algorithm on PDEs *with no semi-analytic solutions*.

Solving PDEs with a neural network as an approximation is a natural idea, and has been considered in various forms previously. [5], [6], [7], [8], and [9] propose to use neural networks to solve PDEs and ODEs. These papers estimate neural network solutions on an a priori fixed mesh. This paper proposes using *deep* neural networks and is *meshfree*, which is key to solving high-dimensional PDEs.

In particular, this paper explores several new innovations. First, we focus on high-dimensional PDEs and apply deep learning advances of the past decade to this problem (deep neural networks instead of shallow neural networks, improved optimization methods for neural networks, etc.). Algorithms for high-dimensional free-boundary PDEs are developed, efficiently implemented, and tested. In particular, we develop an iterative method to address the free-boundary. Secondly, to avoid ever forming a mesh, we sample a sequence of

random spatial points. This produces a meshfree method, which is essential for high-dimensional PDEs. Thirdly, the algorithm incorporates a new computational scheme for the efficient computation of neural network gradients arising from the second derivatives of high-dimensional PDEs.

Recently, [10] and [11] have developed a scheme for solving a class of semilinear PDEs which can be represented as forward-backward stochastic differential equations (FBSDEs) and [12] further develops the algorithm. The algorithm developed in [10, 11, 12] focuses on computing the value of the PDE solution at a single point. The algorithm that we present here can be applied to all PDEs and yields the entire solution of the PDE across all time and space. [3] use a convolutional neural network to solve a large sparse linear system which is required in the numerical solution of the Navier-Stokes PDE. In addition, [4] have recently developed a novel partial differential equation approach to optimize deep neural networks.

[13] also developed an algorithm for the solution of the discrete-time version of the American option free-boundary problem in high dimensions. Their algorithm, commonly called the ‘‘Longstaff-Schwartz method’’, uses dynamic programming and approximates the solution using a separate function approximator at each discrete time (typically a linear combination of basis functions). Our algorithm directly solves the PDE, and uses a single function approximator for all space and all time. The Longstaff-Schwartz algorithm has been further analyzed by [14], [15], and others. Sparse grid methods have also been used to solve high-dimensional PDEs; see [16], [17], [18], [19], and [20].

1.1 Organization of Paper

The deep learning algorithm for solving PDEs is presented in Section 2. An efficient scheme for evaluating the diffusion operator is developed in Section 3. Numerical analysis of the algorithm is presented in Section 4. We implement and test the algorithm on a high-dimensional free-boundary PDE (American options) in up to 200 dimensions. The theorem and proof for the approximation of PDE solutions with neural networks is presented in Section 5. Conclusions are in Section 6.

2 Algorithm

Consider a parabolic PDE with d spatial dimensions:

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \mathcal{L}u(t, x) &= 0, \quad (t, x) \in [0, T] \times \Omega, \\ u(t = 0, x) &= u_0(x), \\ u(t, x) &= g(t, x), \quad x \in \partial\Omega, \end{aligned} \tag{2.1}$$

where $x \in \Omega \subset \mathbb{R}^d$. The DGM algorithm approximates $u(t, x)$ with a deep neural network $f(t, x; \theta)$ where $\theta \in \mathbb{R}^K$ are the neural network’s parameters. Note that the differential operators $\frac{\partial f}{\partial t}(t, x; \theta)$ and $\mathcal{L}f(t, x; \theta)$ can be calculated analytically. Construct the objective function:

$$J(f) = \left\| \frac{\partial f}{\partial t}(t, x; \theta) + \mathcal{L}f(t, x; \theta) \right\|_{[0, T] \times \Omega, \nu_1}^2 + \|f(t, x; \theta) - g(x)\|_{[0, T] \times \partial\Omega, \nu_2}^2 + \|f(0, x; \theta) - u_0(x)\|_{\Omega, \nu_3}^2.$$

Here, $\|f(y)\|_{\mathcal{Y}, \nu}^2 = \int_{\mathcal{Y}} |f(y)|^2 \nu(y) dy$ where $\nu(y)$ is a positive probability density on $y \in \mathcal{Y}$. $J(f)$ measures how well the function $f(t, x; \theta)$ satisfies the PDE differential operator, boundary conditions, and initial condition. If $J(f) = 0$, then $f(t, x; \theta)$ is a solution to the PDE (2.1).

The goal is to find a set of parameters θ such that the function $f(t, x; \theta)$ minimizes the error $J(f)$. If the error $J(f)$ is small, then $f(t, x; \theta)$ will closely satisfy the PDE differential operator, boundary conditions, and initial condition. Therefore, a θ which minimizes $J(f(\cdot; \theta))$ produces a reduced-form model $f(t, x; \theta)$ which approximates the PDE solution $u(t, x)$.

Estimating θ by directly minimizing $J(f)$ is infeasible when the dimension d is large since the integral over Ω is computationally intractable. However, borrowing a machine learning approach, one can instead minimize $J(f)$ using stochastic gradient descent on a sequence of time and space points *drawn at random* from Ω and $\partial\Omega$. This avoids ever forming a mesh.

The DGM algorithm is:

1. Generate random points (t_n, x_n) from $[0, T] \times \Omega$ and (τ_n, z_n) from $[0, T] \times \partial\Omega$ according to respective probability densities ν_1 and ν_2 . Also, draw the random point w_n from Ω with probability density ν_3 .
2. Calculate the squared error $G(\theta_n, s_n)$ at the randomly sampled points $s_n = \{(t_n, x_n), (\tau_n, z_n), w_n\}$ where:

$$G(\theta_n, s_n) = \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}f(t_n, x_n; \theta_n) \right)^2 + \left(f(\tau_n, z_n; \theta_n) - g(z_n) \right)^2 + \left(f(0, w_n; \theta_n) - u_0(w_n) \right)^2.$$

3. Take a descent step at the random point s_n :

$$\theta_{n+1} = \theta_n - \alpha_n \nabla_\theta G(\theta_n, s_n)$$

4. Repeat until convergence criterion is satisfied.

The steps $\nabla_\theta G(\theta_n, s_n)$ are unbiased estimates of $\nabla_\theta J(f(\cdot; \theta_n))$:

$$\mathbb{E}[\nabla_\theta G(\theta_n, s_n) | \theta_n] = \nabla_\theta J(f(\cdot; \theta_n)).$$

The “learning rate” α_n decreases with n . Under (relatively general) technical conditions (see [21]), the algorithm θ_n will converge to a critical point of the objective function $J(f(\cdot; \theta))$ as $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \|\nabla_\theta J(f(\cdot; \theta_n))\| = 0.$$

The DGM algorithm can be trivially modified to apply to hyperbolic, elliptic, and partial-integral differential equations. The algorithm remains essentially the same for these other types of PDEs.

3 A Monte Carlo Method for Fast Computation of Second Derivatives

This section describes a modified algorithm which may be more computationally efficient in some cases. The term $\mathcal{L}f(t, x; \theta)$ contains second derivatives $\frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; \theta)$ which may be expensive to compute in higher dimensions. For instance, 20,000 second derivatives must be calculated in $d = 200$ dimensions. Instead of directly calculating these second derivatives, we approximate the second derivatives using a Monte Carlo method.

Suppose the sum of the second derivatives in $\mathcal{L}f(t, x; \theta)$ is of the form $\frac{1}{2} \sum_{i,j=1}^d \rho_{i,j} \sigma_i(x) \sigma_j(x) \frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; \theta)$, assume $[\rho_{i,j}]_{i,j=1}^d$ is a positive definite matrix, and define $\sigma(x) = (\sigma_1(x), \dots, \sigma_d(x))$. A generalization of the algorithm in this section to second derivatives with nonlinear coefficient dependence on $u(t, x)$ is also possible. Then,

$$\sum_{i,j=1}^d \rho_{i,j} \sigma_i(x) \sigma_j(x) \frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; \theta) = \lim_{\Delta \rightarrow 0} \mathbb{E} \left[\sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x) W_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) W_\Delta^i \right], \quad (3.1)$$

where $W_t \in \mathbb{R}^d$ is a Brownian motion. Define:

$$\begin{aligned} G_1(\theta_n, s_n) &= \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}f(t_n, x_n; \theta_n) \right)^2, \\ G_2(\theta_n, s_n) &= \left(f(\tau_n, z_n; \theta_n) - g(z_n) \right)^2, \\ G_3(\theta_n, s_n) &= \left(f(0, w_n; \theta_n) - u_0(w_n) \right)^2, \\ G(\theta_n, s_n) &= G_1(\theta_n, s_n) + G_2(\theta_n, s_n) + G_3(\theta_n, s_n). \end{aligned}$$

The DGM algorithm use the gradient $\nabla_\theta G_1(\theta_n, s_n)$, which requires the calculation of the second derivative terms in $\mathcal{L}f(t_n, x_n; \theta_n)$. Define the first derivative operators as

$$\mathcal{L}_1 f(t_n, x_n; \theta_n) = \mathcal{L}f(t_n, x_n; \theta_n) - \frac{1}{2} \sum_{i,j=1}^d \rho_{i,j} \sigma_i(x_n) \sigma_j(x_n) \frac{\partial^2 f}{\partial x_i \partial x_j}(t_n, x_n; \theta).$$

Using [\(3.1\)](#), $\nabla_\theta G_1$ is approximated as $\nabla_\theta \tilde{G}_1$ with a fixed constant $\Delta > 0$:

$$\begin{aligned} \nabla_\theta \tilde{G}_1(\theta_n, s_n) &= \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) + \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x)W_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) W_\Delta^i \right) \\ &\times \nabla_\theta \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) + \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x)\tilde{W}_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) \tilde{W}_\Delta^i \right), \end{aligned}$$

where W_Δ is a d -dimensional normal random variable with $\mathbb{E}[W_\Delta] = 0$ and $\text{Cov}[(W_\Delta)_i, (W_\Delta)_j] = \rho_{i,j} \Delta$. $\tilde{W}_\Delta$ has the same distribution as W_Δ . W_Δ and $\tilde{W}_\Delta$ are independent. $\nabla_\theta \tilde{G}_1(\theta_n, s_n)$ is a Monte Carlo approximation of $\nabla_\theta G_1(\theta_n, s_n)$. $\nabla_\theta \tilde{G}_1(\theta_n, s_n)$ has $\mathcal{O}(\sqrt{\Delta})$ bias as an approximation for $\nabla_\theta J_1(f; \theta_n, s_n)$. This approximation error can be further improved via the following scheme using antithetic variates:

$$\begin{aligned} \nabla_\theta \tilde{G}_1(\theta_n, s_n) &= \nabla_\theta \tilde{G}_{1,a}(\theta_n, s_n) + \nabla_\theta \tilde{G}_{1,b}(\theta_n, s_n) \tag{3.2} \\ \nabla_\theta \tilde{G}_{1,a}(\theta_n, s_n) &= \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) + \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x)W_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) W_\Delta^i \right) \\ &\times \nabla_\theta \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) + \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x + \sigma(x)\tilde{W}_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) \tilde{W}_\Delta^i \right), \\ \nabla_\theta \tilde{G}_{1,b}(\theta_n, s_n) &= \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) - \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x - \sigma(x)W_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) W_\Delta^i \right) \\ &\times \nabla_\theta \left(\frac{\partial f}{\partial t}(t_n, x_n; \theta_n) + \mathcal{L}_1 f(t_n, x_n; \theta_n) - \frac{1}{2} \sum_{i=1}^d \frac{\frac{\partial f}{\partial x_i}(t, x - \sigma(x)\tilde{W}_\Delta; \theta) - \frac{\partial f}{\partial x_i}(t, x; \theta)}{\Delta} \sigma_i(x) \tilde{W}_\Delta^i \right). \end{aligned}$$

The approximation [\(3.2\)](#) has $\mathcal{O}(\Delta)$ bias as an approximation for $\nabla_\theta G_1(\theta_n, s_n)$ (the error can be analyzed using a Taylor expansion.) It is important to highlight that there is no computational cost associated with the magnitude of Δ ; an arbitrarily small Δ can be chosen with no additional computational cost (although there may be numerical underflow or overflow problems). The modified algorithm using the Monte Carlo approximation for the second derivatives is:

1. Generate random points (t_n, x_n) from $[0, T] \times \Omega$ and (τ_n, z_n) from $[0, T] \times \partial\Omega$ according to respective densities ν_1 and ν_2 . Also, draw the random point w_n from Ω with density ν_3 .
2. Calculate the descent step $\nabla_\theta G(\theta_n, s_n) = \nabla_\theta \tilde{G}_1(\theta_n, s_n) + \nabla_\theta G_2(\theta_n, s_n) + \nabla_\theta G_3(\theta_n, s_n)$ at the randomly sampled points $s_n = \{(t_n, x_n), (\tau_n, z_n), w_n\}$.
3. Take a descent step at the random point s_n :

$$\theta_{n+1} = \theta_n - \alpha_n \nabla_\theta G(\theta_n, s_n)$$

4. Repeat until convergence criterion is satisfied.

In conclusion, the modified algorithm here is computationally less expensive than the original algorithm in [Section 2](#) but introduces some bias and variance. The original algorithm in [Section 2](#) is unbiased and has lower variance, but is computationally more expensive.

4 Numerical analysis for American Options (a free-boundary PDE)

We test our algorithm on the free-boundary PDE for American options. An American option is a financial derivative on a portfolio of stocks. The option owner may at any time $t \in [0, T]$ choose to exercise the American option and receive a payoff which is determined by the underlying prices of the stocks in the portfolio. T is called the maturity date of the option and the payoff function is $g(x) : \mathbb{R}^d \rightarrow \mathbb{R}$. Let $X_t \in \mathbb{R}^d$ be the prices of d stocks. If at time t the stock prices $X_t = x$, the price of the option is $u(t, x)$. The price function $u(t, x)$ satisfies a free-boundary PDE on $[0, T] \times \mathbb{R}^d$. For American options, one is primarily interested in the solution $u(0, X_0)$ since this is the fair price to buy or sell the option.

Besides the high dimensions and the free-boundary, the American option PDE is challenging to numerically solve since the payoff function $g(x)$ (which both appears in the initial condition and determines the free-boundary) is not continuously differentiable.

Section 4.1 states the free-boundary PDE and the deep learning algorithm to solve it. An iterative method is designed to address the free-boundary. Section 4.2 describes the architecture and implementation details for the neural network. Section 4.3 reports numerical accuracy for a case where a semi-analytic solution exists. Section 4.4 reports numerical accuracy for a case where no semi-analytic solution exists. A method for further improving accuracy via “model triangulation” is presented in Section 4.5.

4.1 The Free-Boundary PDE

We now specify the free-boundary PDE for $u(t, x)$. The stock dynamics and option price are:

$$\begin{aligned} dX_t^i &= \mu(X_t^i)dt + \sigma(X_t^i)dW_t^i, \\ u(t, x) &= \sup_{\tau \geq t} \mathbb{E}[e^{-r(\tau \wedge T)} g(X_{\tau \wedge T}) | X_t = x], \end{aligned}$$

where $W_t \in \mathbb{R}^d$ is a standard Brownian motion and $\text{Cov}[dW_t^i, dW_t^j] = \rho_{i,j}dt$. The price of the American option is $u(0, X_0)$. The price function $u(t, x)$ is the solution to a free-boundary PDE and will satisfy:

$$\begin{aligned} 0 &= \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \sum_{i,j=1}^d \rho_{i,j} \sigma(x_i) \sigma(x_j) \frac{\partial^2 u}{\partial x_i \partial x_j}(t, x) - ru(t, x), \quad \forall \{(t, x) : u(t, x) > g(x)\}. \\ u(t, x) &\geq g(x), \quad \forall (t, x). \\ u(t, x) &\in C^1(\mathbb{R}_+ \times \mathbb{R}^d), \quad \forall \{(t, x) : u(t, x) = g(x)\}. \\ u(T, x) &= g(x), \quad \forall x. \end{aligned} \tag{4.1}$$

The free-boundary set is $F = \{(t, x) : u(t, x) = g(x)\}$. $u(t, x)$ satisfies a partial differential equation “above” the free-boundary set F , and $u(t, x)$ equals the function $g(x)$ “below” the free-boundary set F .

The deep learning algorithm for solving the PDE (4.1) requires simulating points above and below the free-boundary set F . We use an iterative method to address the free-boundary. The free-boundary set F is approximated using the current parameter estimate θ_n . This approximate free-boundary is used in the probability measure that we simulate points with. The gradient is not taken with respect to the θ_n input of the probability density used to simulate random points. For this purpose, define the objective function:

$$\begin{aligned} J(f; \theta, \tilde{\theta}) &= \left\| \frac{\partial f}{\partial t}(t, x; \theta) + \frac{1}{2} \sum_{i,j=1}^d \rho_{i,j} \sigma(x_i) \sigma(x_j) \frac{\partial^2 f}{\partial x_i \partial x_j}(t, x; \theta) - rf(t, x; \theta) \right\|_{[0,T] \times \Omega, \nu_1(\tilde{\theta})}^2 \\ &+ \|\max(g(x) - f(t, x; \theta), 0)\|_{[0,T] \times \Omega, \nu_2(\tilde{\theta})}^2 \\ &+ \|f(T, x; \theta) - g(x)\|_{\Omega, \nu_3}^2. \end{aligned}$$

Descent steps are taken in the direction $-\nabla_{\theta} J(f; \theta, \tilde{\theta})$. $\nu_1(\tilde{\theta})$ and $\nu_2(\tilde{\theta})$ are the densities of the points in $\tilde{B}^1$ and $\tilde{B}^2$, which are defined below. The deep learning algorithm is:

1. Generate the random batch of points $B^1 = \{t_m, x_m\}_{m=1}^M$ from $[0, T] \times \Omega$ according to the probability density ν_1^0 . Select the points $\tilde{B}^1 = \{(t, x) \in B^1 : f(t, x; \theta_n) > g(x)\}$.

2. Generate the random batch of points $B^2 = \{\tau_m, z_m\}_{m=1}^M$ from $[0, T] \times \partial\Omega$ according to the probability density ν_2^0 . Select the points $\tilde{B}^2 = \{(\tau, z) \in B^2 : f(\tau, z; \theta_n) \leq g(z)\}$.
3. Generate the random batch of points $B^3 = \{w_m\}_{m=1}^M$ from Ω with probability density ν_3 .
4. Approximate $J(f; \theta_n, \tilde{\theta}_n)$ as $J(f; \theta_n, S_n)$ at the randomly sampled points $S_n = \{\tilde{B}^1, \tilde{B}^2, B^3\}$:

$$\begin{aligned}
J(f; \theta_n, S_n) &= \frac{1}{|\tilde{B}^1|} \sum_{(t_m, x_m) \in \tilde{B}^1} \left(\frac{\partial f}{\partial t}(t_m, x_m; \theta_n) + \frac{1}{2} \sum_{i,j=1}^d \rho_{i,j} \sigma(x_i) \sigma(x_j) \frac{\partial^2 f}{\partial x_i \partial x_j}(t_m, x_m; \theta_n) - r f(t_m, x_m; \theta_n) \right) \\
&+ \frac{1}{|\tilde{B}^2|} \sum_{(\tau_m, z_m) \in \tilde{B}^2} \max(g(z_m) - f(\tau_m, z_m; \theta_n), 0)^2 \\
&+ \frac{1}{|B^3|} \sum_{w_m \in B^3} \left(f(T, w_m; \theta) - g(w_m) \right)^2.
\end{aligned}$$

5. Take a descent step for the random batch S_n :

$$\theta_{n+1} = \theta_n - \alpha_n \nabla_{\theta} J(f; \theta_n, S_n).$$

6. Repeat until convergence criterion is satisfied.

4.2 Architecture and implementation

The hidden units are smooth (e.g., tanh or sigmoid). Consequently, $f(t, x; \theta)$ is smooth and therefore can solve for a “classical solution” of the PDE. The architecture is similar to an LSTM network, using several element-wise multiplications of hidden layers. The only input to the network is (t, x) . We do not use any custom-designed nonlinear transformations of (t, x) . If properly chosen, such additional inputs might help performance. For example, the European option PDE solution (which has an analytic formula) could be included as an input.

The second derivatives are approximated using the method from Section 3. Training is distributed across 6 GPU nodes using asynchronous stochastic gradient descent. Parameters are updated using the ADAM algorithm. The learning rate is decayed over time. Accuracy can be improved by calculating a running average of the neural network solutions over a sequence of training iterations (essentially a computationally cheap approach for building a model ensemble).

4.3 An American Option PDE with a Semi-Analytic Solution

We implement our deep learning algorithm to solve the PDE (4.1). The accuracy of our deep learning algorithm is evaluated in up to 200 dimensions. The results are reported below in Table 1.

Number of dimensions	Error
3	0.05%
20	0.03%
100	0.11%
200	0.22%

Table 1: The deep learning algorithm solution is compared with a semi-analytic solution for the Black-Scholes model. The parameters $\mu(x) = (r - c)x$ and $\sigma(x) = \sigma x$. All stocks are identical with correlation $\rho_{i,j} = .75$, volatility $\sigma = .25$, initial stock price $X_0 = 1$, dividend rate $c = 0.02$, and interest rate $r = 0$. The maturity of the option is $T = 2$ and the strike price is $K = 1$. The payoff function is $g(x) = \max((\prod_{i=1}^d x_i)^{1/d} - K, 0)$. The error is reported for the price $u(0, X_0)$ of the at-the-money American call option. The error is $\frac{|f(0, X_0; \theta) - u(0, X_0)|}{|u(0, X_0)|} \times 100\%$.

Although the solution at $(t, x) = (0, X_0)$ is of primary interest for American options, most other PDE applications are interested in the entire solution $u(t, x)$. The deep learning algorithm provides an approximate solution across all time and space $(t, x) \in [0, T] \times \Omega$. As an example, we present in Figure 1 contour plots of the absolute error and percent error across time and space for the American option PDE in 20 dimensions. The algorithm samples time points uniformly at random on $[0, T]$ and samples space points from the joint distribution of $X_t^1, \dots, X_t^d$ in equation (4.1). This produces an “envelope” of sampled points since X_t spreads out as a diffusive process from $X_0 = 1$. The y-axis of the contour plots is the geometric average $(\prod_{i=1}^d x_i)^{1/d}$, which corresponds to the final condition $g(x)$. The percent error $\frac{|f(t, x; \theta) - u(t, x)|}{|u(t, x)|} \times 100\%$ is reported for points where $|u(t, x)| > 0.05$. The absolute error becomes relatively large in a few areas; however, the solution $u(t, x)$ also grows large in these areas and therefore the percent error remains small.

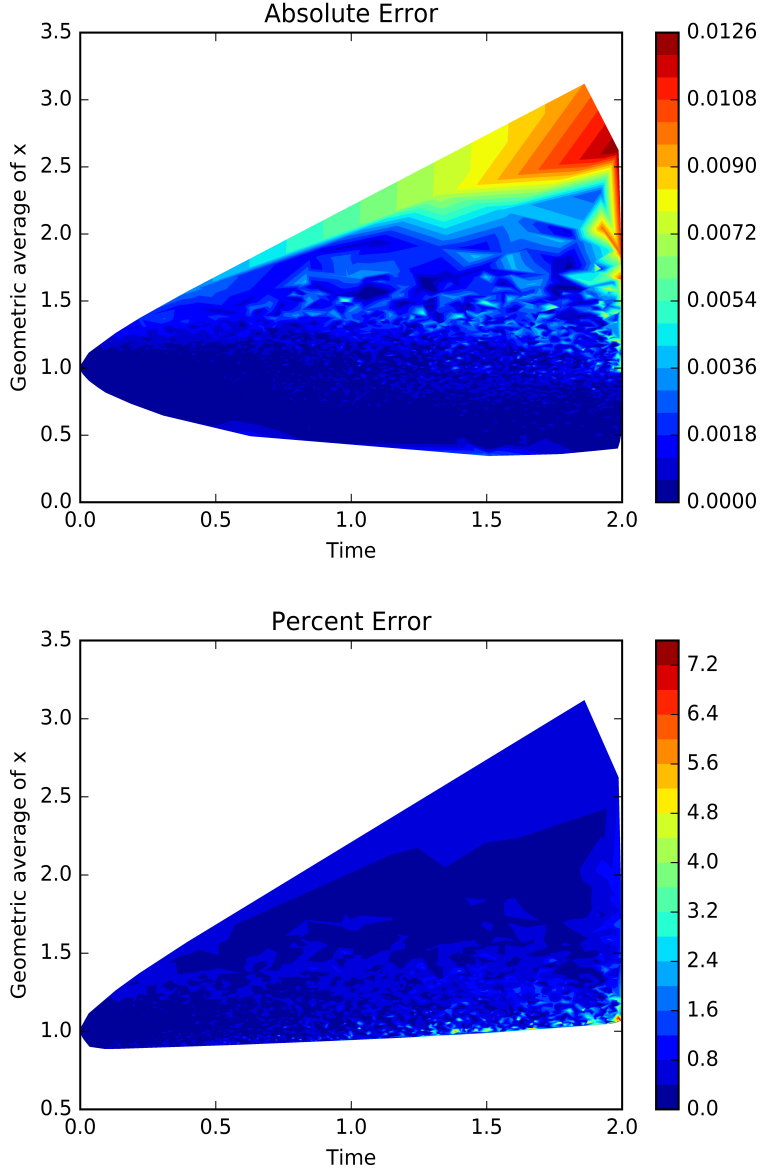


Figure 1: Top: Absolute error. Bottom: Percent error. For reference, the price at time 0 is 0.1003 and the solution at time T is $\max(\text{geometric average of } x - 1, 0)$.

4.4 An American Option PDE *without* a Semi-Analytic Solution

We now consider a case of the American option PDE which does not have a semi-analytic solution. The American option PDE has the special property that it is possible to calculate error bounds on an approximate solution. Therefore, we can evaluate the accuracy of the deep learning algorithm even on cases where no semi-analytic solution is available.

We previously only considered a symmetrical case where $\rho_{i,j} = 0.75$ and $\sigma = 0.25$ for all stocks. This section solves a more challenging heterogeneous case where $\rho_{i,j}$ and σ_i vary across all dimensions $i = 1, 2, \dots, d$. The coefficients are fitted to actual data for the stocks IBM, Amazon, Tiffany, Amgen, Bank of America, General Mills, Cisco, Coca-Cola, Comcast, Deere, General Electric, Home Depot, Johnson & Johnson, Morgan Stanley, Microsoft, Nordstrom, Pfizer, Qualcomm, Starbucks, and Tyson Foods from 2000-2017. This produces a PDE with widely-varying coefficients for each of the $\frac{d^2+d}{2}$ second derivative terms. The correlation coefficients $\rho_{i,j}$ range from -0.53 to 0.80 for $i \neq j$ and σ_i ranges from 0.09 to 0.69 .

Let $f(t, x; \theta)$ be the neural network approximation. [14] derived that the PDE solution $u(t, x)$ lies in the interval:

$$\begin{aligned} u(t, x) &\in [\underline{u}(t, x), \bar{u}(t, x)], \\ \underline{u}(t, x) &= \mathbb{E} \left[g(X_\tau) | X_t = x, \tau > t \right], \\ \bar{u}(t, x) &= \mathbb{E} \left[\sup_{s \in [t, T]} [e^{-r(s-t)} g(X_s) - M_s] \right]. \end{aligned}$$

where $\tau = \inf\{t \in [0, T] : f(t, X_t; \theta) < g(X_t)\}$ and M_s is a martingale constructed from the approximate solution $f(t, x; \theta)$

$$M_s = f(s, X_s; \theta) - f(t, X_t; \theta) - \int_t^s \left[\frac{\partial f}{\partial t}(s', X_{s'}; \theta) + \frac{1}{2} \sum_{i,j=1}^d \sigma(X_{s',i}) \sigma(X_{s',j}) \frac{\partial^2 f}{\partial x_i \partial x_j}(s', X_{s'}; \theta) - r f(s', X_{s'}; \theta) \right] ds'.$$

The bounds (4.2) depend only on the approximation $f(t, x; \theta)$, which is known, and can be evaluated via Monte Carlo simulation. The integral for M_s must also be discretized. The best estimate for the price of the American option is the midpoint of the interval $[\underline{u}(0, X_0), \bar{u}(0, X_0)]$, which has an error bound of $\frac{\bar{u}(0, X_0) - \underline{u}(0, X_0)}{2\underline{u}(0, X_0)} \times 100\%$. Numerical results are in Table 2.

Strike price	Neural network solution	Lower Bound	Upper Bound	Error bound
0.90	0.14856	0.14838	0.14887	0.17%
0.95	0.12281	0.12270	0.12327	0.23%
1.00	0.10153	0.10119	0.10193	0.37%
1.05	0.08341	0.08315	0.08381	0.40%
1.10	0.06835	0.06809	0.06892	0.60%

Table 2: The accuracy of the deep learning algorithm is evaluated on a case where there is no semi-analytic solution. The parameters $\mu(x) = (r - c)x$ and $\sigma(x) = \sigma x$. The correlations $\rho_{i,j}$ and volatilities σ_i are estimated from data to generate a heterogeneous diffusion matrix. The initial stock price is $X_0 = 1$, dividend rate $c = 0.02$, and interest rate $r = 0$ for all stocks. The maturity of the option is $T = 2$. The payoff function is $g(x) = \max(\frac{1}{d} \sum_{i=1}^d x_i - K, 0)$. The neural network solution and its error bounds are reported for the price $u(0, X_0)$ of the American call option. The best estimate for the price of the American option is the midpoint of the interval $[\underline{u}(0, X_0), \bar{u}(0, X_0)]$, which has an error bound of $\frac{\bar{u}(0, X_0) - \underline{u}(0, X_0)}{2\underline{u}(0, X_0)} \times 100\%$. In order to calculate the upper bound, the integral (4.2) is discretized with time step size $\Delta = 10^{-2}$.

We present in Figure 2 contour plots of the absolute error bound and percent error bound across time and space for the American option PDE in 20 dimensions for strike price $K = 1$. The y-axis of the contour plots is

the average $\frac{1}{d} \sum_{i=1}^d x_i$, which corresponds to the final condition $g(x)$. The percent error $\frac{|f(t,x;\theta) - u(t,x)|}{|u(t,x)|} \times 100\%$ is reported for points where $|u(t,x)| > 0.05$. It should be emphasized that these are *error bounds*; therefore, the actual error could be lower. The contour plot [2](#) requires significant computations. For each point at which calculate an error bound, a new simulation of (4.2) is required. In total, almost 20 billion paths are simulated, which we distribute across hundreds of GPUs on the Blue Waters supercomputer.

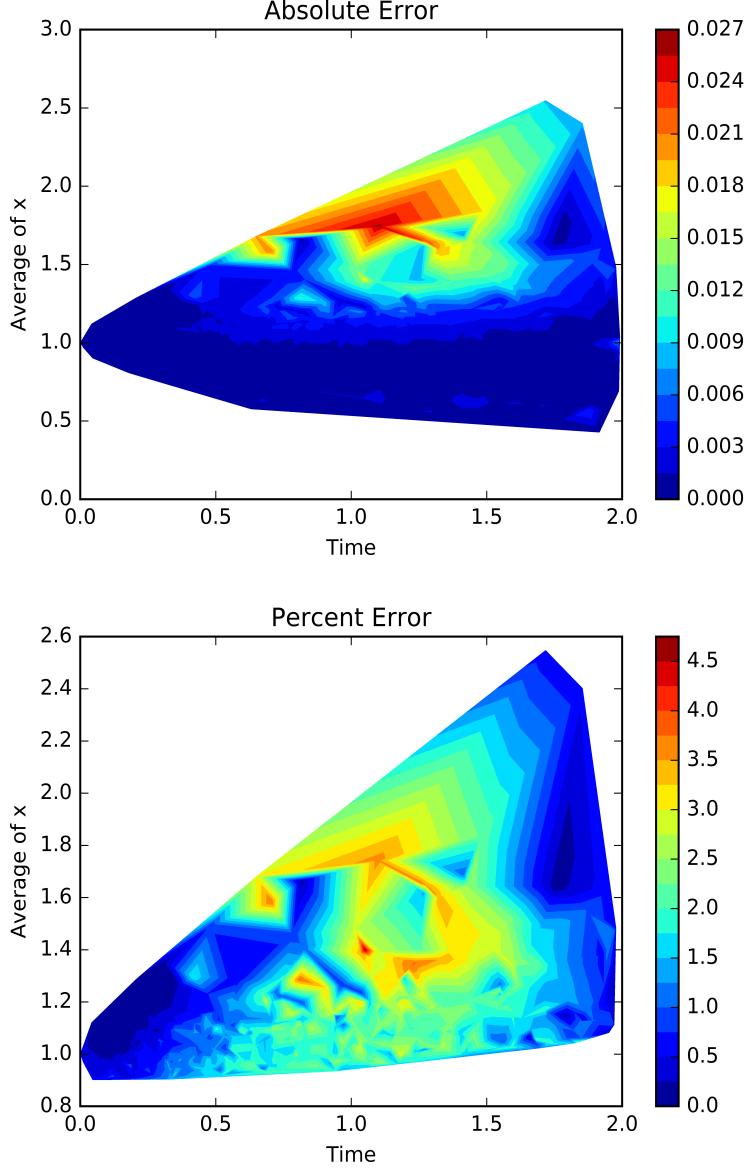


Figure 2: Top: Absolute error. Bottom: Percent error. For reference, $u(0, X_0) \in [0.10119, 0.10193]$ and the solution at time T is $\max(\text{average of } x - 1, 0)$.

4.5 Model Triangulation

We also develop a method using an ensemble of models to further improve the accuracy for American option pricing. Neural networks are non-convex and stochastic gradient descent typically finds different local minima

each time a model is trained. This will produce different upper and lower bounds on the solution $u(t, x)$. A number of neural networks are trained, each with a different random initialization and a different random sequence of training points. Each neural network provides an upper and lower bound on the solution $u(t, x)$. The largest lower bound and smallest upper bound are chosen from all of the neural networks. We call this method “model triangulation”. Let the bounds from the m -th neural network be:

$$u(t, x) \in [\underline{u}_m(t, x), \bar{u}_m(t, x)], \quad m = 0, 1, 2, \dots, M.$$

Then, $u(t, x)$ is bounded by:

$$\begin{aligned} u(t, x) &\in [\underline{u}_*(t, x), \bar{u}_*(t, x)], \quad m = 0, 1, 2, \dots, M. \\ \underline{u}_*(t, x) &= \max_{m \in 0, \dots, M} \underline{u}_m(t, x). \\ \bar{u}_*(t, x) &= \min_{m \in 0, \dots, M} \bar{u}_m(t, x), \end{aligned}$$

which is superior to the bounds provided by any individual neural network. The best estimate for the American option price is $\bar{u}_*(0, X_0) + \frac{1}{2}(\bar{u}_*(0, X_0) - \underline{u}_*(0, X_0))$, which has error bound $100\% \times \frac{\bar{u}_*(0, X_0) - \underline{u}_*(0, X_0)}{2\underline{u}_*(0, X_0)}$. Numerical results are in Table 3

Ave. neural network solution	Ave. error bound	Error bound with model triangulation
0.10151	0.37%	0.17%

Table 3: The American option PDE is the same as in Table 2 (it has no semi-analytic solution). 20 neural networks are trained and the average solution for the price is reported in the first column. The best estimate for the price and the corresponding error bound is calculated for each model. The average of these error bounds is reported in the second column. The third column reports the error bound for the estimate of the price using model triangulation.

5 Neural Network Approximation Theorem for PDEs

Let the L^2 error $J(f)$ measure how well the neural network f satisfies the differential operator, boundary condition, and initial condition. Define $\mathfrak{C}^n$ as the class of neural networks with n hidden units and let f_n be a neural network with n hidden units which minimizes $J(f)$. We prove that

$$\begin{aligned} f_n &= \arg \min_{f \in \mathfrak{C}^n} J(f), \\ f_n &\rightarrow u \quad \text{as } n \rightarrow \infty \end{aligned}$$

in the appropriate sense, for quasilinear parabolic PDE’s with the principle term in divergence form under certain growth assumptions on the nonlinear terms.

The proof requires the joint analysis of the approximation power of neural networks as well as the continuity properties of partial differential equations. First, we show that the neural network can satisfy the differential operator, boundary condition, and initial condition arbitrarily well for sufficiently large n .

$$J(f_n) \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

Let u be the solution to the PDE. The statement (5.1) does not necessarily imply that $f_n \rightarrow u$. One challenge to proving convergence is that we only have L^2 control of the error. We prove convergence by first establishing that each neural network $\{f_n\}_{n=1}^\infty$ satisfies a PDE with a source term $h^n(t, x)$. Importantly, the source terms $h^n(t, x)$ are only known to be bounded in L^2 norm. We are then able to prove the convergence of $f_n \rightarrow u$ as $n \rightarrow \infty$ using compactness arguments.

The precise statement of the theorem and the presentation of the proof is in the next two sections. Section 5.1 proves that $J(f_n) \rightarrow 0$ as $n \rightarrow \infty$. Section 5.2 proves convergence of f_n to the solution u of the PDE as $n \rightarrow \infty$. The main result is Theorem 5.4

5.1 Convergence of the L^2 error $J(f)$

In this section, we present a theorem guaranteeing the existence of multilayer feed forward networks able to universally approximate solutions of quasilinear PDEs. To do so, we use the results of [26] on universal approximation of functions and their derivatives and make appropriate assumptions on the coefficients of the PDEs to guarantee that a classical solution exists (since then the results of [26] apply).

Consider a bounded set $\Omega \subset \mathbb{R}^d$ with a smooth boundary $\partial\Omega$ and denote $\Omega_T = (0, T] \times \Omega$ and $\partial\Omega_T = (0, T] \times \partial\Omega$. We consider the class of semilinear parabolic PDE's of the form

$$\begin{aligned} \partial_t u(t, x) - \operatorname{div}(\alpha(t, x, u(t, x), \nabla u(t, x))) + \gamma(t, x, u(t, x), \nabla u(t, x)) &= 0, \text{ for } (t, x) \in \Omega_T \\ u(0, x) &= u_0(x), \text{ for } x \in \Omega \\ u(t, x) &= g(t, x), \text{ for } (t, x) \in \partial\Omega_T \end{aligned} \quad (5.1)$$

For notational convenience, let us write the operator of (5.1) as $\mathcal{G}$. Namely, let us denote

$$\mathcal{G}[u](t, x) = \partial_t u(t, x) - \operatorname{div}(\alpha(t, x, u(t, x), \nabla u(t, x))) + \gamma(t, x, u(t, x), \nabla u(t, x)).$$

Notice that we can write

$$\mathcal{G}[u](t, x) = \partial_t u(t, x) - \sum_{i,j=1}^d \frac{\partial \alpha_i(t, x, u(t, x), \nabla u(t, x))}{\partial u_{x_j}} \partial_{x_i, x_j} u(t, x) + \hat{\gamma}(t, x, u(t, x), \nabla u(t, x)),$$

where

$$\hat{\gamma}(t, x, u, p) = \gamma(t, x, u, p) - \sum_{i=1}^d \frac{\partial \alpha_i(t, x, u, p)}{\partial u} \partial_{x_i} u - \sum_{i=1}^d \frac{\partial \alpha_i(t, x, u, p)}{\partial x_i}.$$

For the purposes of this section, we consider equations of the type (5.1) that have classical solutions. In particular we assume that there is a unique $u(t, x)$ solving (5.1) such that

$$u(t, x) \in \mathcal{C}(\bar{\Omega}_T) \cap \mathcal{C}^{1+\beta/2, 2+\beta}(\Omega_T) \text{ and that } \sup_{(t,x) \in \Omega_T} |\nabla_x u(t, x)| < \infty. \quad (5.2)$$

We refer the interested reader to Theorem 6.2 of Chapter V or to Theorem 4.2 of Chapter IV of [27] for specific general conditions on α, γ guaranteeing the validity of the aforementioned statement.

Universal approximation results for single functions and their derivatives have been obtained under various assumptions in [24, 25, 26]. In this paper, we use a result obtained in [26]. Let us recall the setup appropriately modified for our case of interest. Let ψ be an activation function of the hidden units and define the set

$$\mathfrak{C}^n(\psi) = \left\{ h(t, x) : \mathbb{R}^{1+d} \mapsto \mathbb{R} : h(t, x) = \sum_{i=1}^n \beta_i \psi \left(\alpha_{1,i} t + \sum_{j=1}^d \alpha_{j,i} x_j + c_j \right) \right\}. \quad (5.3)$$

Let the parameter be $\theta = (\beta_1, \dots, \beta_n, \alpha_{1,1}, \dots, \alpha_{d,n}, c_1, c_1, \dots, c_n) \in \mathbb{R}^{2n+n(1+d)}$. Then we have the following result.

Theorem 5.1. *Let $\mathfrak{C}^n(\psi)$ be given by (5.3) where ψ is assumed to be bounded and non-constant. Set $\mathfrak{C}(\psi) = \bigcup_{n \geq 1} \mathfrak{C}^n(\psi)$. Assume that Ω_T is compact and consider the measures ν_1, ν_2, ν_3 whose support is contained in Ω_T, Ω and $\partial\Omega_T$ respectively. In addition, assume that the PDE (5.1) has a unique classical solution such that (5.2) holds. Also, assume that the nonlinear terms $\frac{\partial \alpha_i(t, x, u, p)}{\partial p_j}$ and $\hat{\gamma}(t, x, u, p)$ are locally Lipschitz in (u, p) with Lipschitz constant that can have at most polynomial growth on u and p . Then, for every $\epsilon > 0$, there exists a positive constant $K > 0$ that may depend on $\sup_{\Omega_T} \sum_{i,j=1}^d \left| \frac{\partial \alpha_i(t, x, u, \nabla u)}{\partial u_{x_j}} \right|$, $\sup_{\Omega_T} |\hat{\gamma}(t, x, u, \nabla u)|$, $\sup_{\Omega_T} |u|$ and $\sup_{\Omega_T} |\nabla_x u|$ such that there exists a function $f \in \mathfrak{C}(\psi)$ that satisfies*

$$J(f) \leq K\epsilon.$$

Proof of Theorem 5.1 By Theorem 3 of [26] we know that there is a function $f \in \mathfrak{C}(\psi)$ that is uniformly 2-dense on compacts of $\mathcal{C}^2(\mathbb{R}^{1+d})$. This means that for $u \in \mathcal{C}^{1,2}([0, T] \times \mathbb{R}^d)$ and $\epsilon > 0$, there is $f \in \mathfrak{C}(\psi)$ such that

$$\sup_{(t,x) \in \Omega_T} |\partial_t u(t, x) - \partial_t f(t, x; \theta)| + \max_{|a| \leq 2} \sup_{(t,x) \in \Omega_T} |\partial_x^{(a)} u(t, x) - \partial_x^{(a)} f(t, x; \theta)| < \epsilon \quad (5.4)$$

Notice now that due to the locally Lipschitz property of the nonlinearity $\hat{\gamma}$ we have, using Hölder inequality with exponents r_1, r_2 ,

$$\begin{aligned} & \int_{\Omega_T} |\hat{\gamma}(t, x, f, \nabla_x f) - \hat{\gamma}(t, x, u, \nabla_x u)|^2 d\nu_1(t, x) \leq \\ & \leq \int_{\Omega_T} (|f(t, x; \theta)|^{q_1} + |\nabla_x f(t, x; \theta)|^{q_2} + |u(t, x)|^{q_3} + |\nabla_x u(t, x)|^{q_4}) \\ & \quad \times (|f(t, x; \theta) - u(t, x)|^2 + |\nabla_x f(t, x; \theta) - \nabla_x u(t, x)|^2) d\nu_1(t, x) \\ & \leq \left(\int_{\Omega_T} (|f(t, x; \theta)|^{q_1} + |\nabla_x f(t, x; \theta)|^{q_2} + |u(t, x)|^{q_3} + |\nabla_x u(t, x)|^{q_4})^{r_1} d\nu_1(t, x) \right)^{1/r_1} \\ & \quad \times \left(\int_{\Omega_T} (|f(t, x; \theta) - u(t, x)|^2 + |\nabla_x f(t, x; \theta) - \nabla_x u(t, x)|^2)^{r_2} d\nu_1(t, x) \right)^{1/r_2} \\ & \leq K \left(\sum_{i=1}^4 \epsilon^{q_i} + \sup_{\Omega_T} |u| + \sup_{\Omega_T} |\nabla_x u| \right) \epsilon^2 \end{aligned} \quad (5.5)$$

In the last step we used (5.4). Similarly, denoting for convenience

$$\beta(t, x, u, \nabla u, \nabla^2 u) = \sum_{i,j=1}^d \frac{\partial \alpha_i(t, x, u(t, x), \nabla u(t, x))}{\partial u_{x_j}} \partial_{x_i, x_j} u(t, x),$$

we obtain that for some constant $K = K(\sup_{\Omega_T} |u|, \sup_{\Omega_T} |\nabla_x u|)$

$$\int_{\Omega_T} |\beta(t, x, f, \nabla_x f, \nabla_x^2 f) - \beta(t, x, u, \nabla_x u, \nabla_x^2 u)|^2 d\nu_1(t, x) \leq K \epsilon^2 \quad (5.6)$$

Using (5.4) and (5.5)-(5.6) we subsequently obtain for the objective function

$$\begin{aligned} J(f) &= \left\| \frac{\partial f}{\partial t}(t, x; \theta) + \beta(t, x, f, \nabla f, \nabla^2 f) + \hat{\gamma}(t, x, f, \nabla_x f) \right\|_{\Omega_T, \nu_1}^2 + \|f(t, x; \theta) - g(t, x)\|_{\partial \Omega_T, \nu_2}^2 + \\ & \quad + \|f(0, x; \theta) - u_0(x)\|_{\Omega, \nu_3}^2 \\ & \leq \int_{\Omega_T} |\partial_t u(t, x) - \partial_t f(t, x; \theta)|^2 d\nu_1 + \int_{\Omega_T} |\beta(t, x, f, \nabla f, \nabla^2 f) - \beta(t, x, u, \nabla u, \nabla^2 u)|^2 d\nu_1 \\ & \quad + \int_{\Omega_T} |\gamma(t, x, f, \nabla_x f) - \gamma(t, x, u, \nabla_x u)|^2 d\nu_1 + \int_{\partial \Omega_T} |f(t, x; \theta) - u(t, x)|^2 d\nu_2 + \\ & \quad + \int_{\Omega} |f(0, x; \theta) - u(0, x)|^2 d\nu_3 \\ & \leq K \epsilon^2 \end{aligned}$$

for an appropriate constant $K < \infty$. The last step completes the proof of the Theorem after rescaling ϵ . $\square$

5.2 Convergence of the neural network to the PDE solution

We now prove the convergence of the neural networks f_n to the solution u of the PDE (5.1) as $n \rightarrow \infty$. The objective function is

$$J(f) = \|\mathcal{G}[f]\|_{2,\Omega_T}^2 + \|f - g\|_{2,\partial\Omega_T}^2 + \|f(0, \cdot) - u_0\|_{2,\Omega}^2$$

Recall that the norms above are $L^2(X)$ norms in the respective space $X = \Omega_T, \partial\Omega_T$ and Ω respectively. From Theorem 5.1, we have that

$$J(f^n) \rightarrow 0 \text{ as } n \rightarrow \infty.$$

Each neural network f^n satisfies the PDE

$$\begin{aligned} \mathcal{G}[f^n](t, x) &= h^n(t, x), \text{ for } (t, x) \in \Omega_T \\ f^n(0, x) &= u_0^n(x), \text{ for } x \in \Omega \\ f^n(t, x) &= g^n(t, x), \text{ for } (t, x) \in \partial\Omega_T \end{aligned} \tag{5.7}$$

for some h^n, u_0^n , and g^n such that

$$\|h^n\|_{2,\Omega_T}^2 + \|g^n - g\|_{2,\partial\Omega_T}^2 + \|u_0^n - u_0\|_{2,\Omega}^2 \rightarrow 0 \text{ as } n \rightarrow \infty. \tag{5.8}$$

Let us first make the following set of assumptions.

Condition 5.2. • There is a constant $\mu > 0$ and positive functions $\kappa(t, x), \lambda(t, x)$ such that for all $(t, x) \in \Omega_T$ we have

$$\|\alpha(t, x, u, p)\| \leq \mu(\kappa(t, x) + |u| + \|p\|), \text{ and } |\gamma(t, x, u, p)| \leq \lambda(t, x)(1 + \|p\|).$$

with $\kappa \in L^2(\Omega_T)$ and $\lambda \in L^{d+2}(\Omega_T)$.

- $\alpha(t, x, u, p)$ and $\gamma(t, x, u, p)$ are continuous in $(u, p) \in \mathbb{R} \times \mathbb{R}^d$ for every $(t, x) \in \Omega_T$.
- $\alpha(t, x, u, p)$ and $\gamma(t, x, u, p)$ are measurable in Ω_T for every $(u, p) \in \mathbb{R} \times \mathbb{R}^d$.
- $\alpha(t, x, u, p)$ and $\gamma(t, x, u, p)$ are Lipschitz in $(u, p) \in \mathbb{R} \times \mathbb{R}^d$ uniformly on compacts of the form $\{(t, x) \in \Omega_T, |u| \leq C, |p| \leq C\}$.
- There is a positive constant $\nu > 0$ such that

$$\alpha(t, x, u, p)p \geq \nu|p|^2$$

and

$$\langle \alpha(t, x, u, p_1) - \alpha(t, x, u, p_2), p_1 - p_2 \rangle > 0, \text{ for every } p_1, p_2 \in \mathbb{R}^d, p_1 \neq p_2.$$

- $g(t, x) \in \mathcal{C}(\bar{\Omega}_T) \cap L^2(\partial\Omega_T)$, $u_0(x) \in L^2(\Omega)$.

Remark 5.3. Theorem 5.4 states a convergence result under Condition 5.2. This condition is being made in order to guarantee uniform boundedness of f^n satisfying (5.7)-(5.8) with respect to n in an appropriate Sobolev space, in particular we chose the space $L^2(0, T; W^{1,2}(\Omega))$. We remark here that due to the assumption of L^2 data and source term, the needed a-priori energy bound needed for the solution to (5.7)-(5.8) in order to be able to justify convergence, depends heavily on the nonlinearity. But as soon as one has a-priori uniform bounds, Theorem 5.4 applies and guarantees convergence.

Theorem 5.4. Consider the problem (5.1) and assume that it has a unique weak solution $u \in W^{1,2}(\Omega_T)$ with Condition 5.2 holding. Then, f^n converges to u , the unique weak solution to (5.1), strongly in $L^1(\Omega_T)$ and in $L^\rho(0, T; W^{1,\rho}(\Omega))$ for every $\rho < 2$.

Proof of Theorem 5.4 The proof follows by compactness arguments. First, we notice that we can assume, without loss of generality, that the boundary term is zero, i.e. $g(t, x) = 0$. Indeed, we can reduce the nonhomogeneous boundary conditions on $\partial\Omega_T$ to the homogeneous one by introducing in place of u the new function $u - \psi$, where ψ is some sufficiently smooth function equal to g on $\partial\Omega_T$, see also [27] for details. So, let us assume from now on that $g = 0$. For the same reasons, we can also assume that $g^n = 0$.

Due to Condition 5.2 Theorem 2.1 [30] implies that f^n is uniformly bounded with respect to n in $L^2(0, T; W^{1,2}(\Omega))$. In regards to such results we also refer the reader to Theorem 2.1 and Remark 2.14 of [23] for the case $\gamma = 0$ and to [28, 29] for related results in the general case.

This uniform energy bound implies that we can extract a subsequence, denoted also by f^n , which converges weakly to some u in $L^2(0, T; W^{1,2}(\Omega))$. Moreover, using the growth assumptions on $\alpha(\cdot)$ and $\gamma(\cdot)$ from Condition 5.2 we get from the equation itself that $\partial_t f^n$ is bounded uniformly with respect to n in $L^1(\Omega_T) + L^2(0, T; W^{-1,2}(\Omega))$. Then, by the compactness arguments of [31] we get that f^n converges strongly to u in $L^1(\Omega_T)$. This means that it converges almost everywhere to u in Ω_T . In addition, the results of [31] also show that the sequence $\{f^n, n \geq 0\}$ is compact in $\mathcal{C}([0, T]; W^{-1,1}(\Omega))$ which means that initial conditions also pass to the limit as $n \rightarrow \infty$.

The nonlinearity of the α and γ functions with respect to the gradient prohibits us from passing to the limit directly in the respective weak formulation. However, by Theorem 3.3 of [22] we get that

$$\nabla f^n \rightarrow \nabla u \text{ almost everywhere in } \Omega_T.$$

Hence, we obtain that f^n converges to u strongly also in $L^\rho(0, T; W^{1,\rho}(\Omega))$ for every $\rho < 2$. In addition, we obtain that

$$\alpha(t, x, f^n, \nabla f^n) \rightarrow \alpha(t, x, u, \nabla u) \text{ strongly in } L^\rho(\Omega_T)$$

and, by the assumed growth conditions,

$$\gamma(t, x, f^n, \nabla f^n) \rightarrow \gamma(t, x, u, \nabla u) \text{ strongly in } L^\rho(\Omega_T)$$

as $n \rightarrow \infty$, for every $\rho < 2$. The weak formulation of the PDE (5.7) with $g^n = 0$ reads as follows. For every $t_1 \in (0, T]$

$$\begin{aligned} & \int_{\Omega_{t_1}} [-f^n \partial_t \phi + \langle \alpha(t, x, f^n, \nabla f^n), \nabla \phi \rangle + (\gamma(t, x, f^n, \nabla f^n) - h^n) \phi] (t, x) dx dt \\ & + \int_{\Omega} f^n(t_1, x) \phi(t_1, x) dx - \int_{\Omega} u_0^n(x) \phi(0, x) dx = 0 \end{aligned}$$

for every $\phi \in C^\infty(\Omega_T)$ that is zero on $\partial\Omega_T$. Using the above convergence results, we then obtain that the limit point u satisfies for every $t_1 \in (0, T]$ the equation

$$\begin{aligned} & \int_{\Omega_{t_1}} [-u \partial_t \phi + \langle \alpha(t, x, u, \nabla u), \nabla \phi \rangle + \gamma(t, x, u, \nabla u) \phi] (t, x) dx dt \\ & + \int_{\Omega} u(t_1, x) \phi(t_1, x) dx - \int_{\Omega} u_0(x) \phi(0, x) dx = 0, \end{aligned}$$

which is the weak formulation of the equation (5.1) with $g = 0$, concluding the proof of the theorem. $\square$

It is clear that the main property driving Theorem 5.4 is that f^n satisfying (5.7) is uniformly bounded with respect to n in $L^2(0, T; W^{1,2}(\Omega))$. In general, such a result depends heavily on the form of the nonlinearities.

In the linear case, an even stronger result holds as described in Example 5.5

Example 5.5 (Linear case). Let us assume that the operator $\mathcal{G}$ is linear in u and ∇u . In particular, let us set

$$\alpha_i(t, x, u, p) = \sum_{j=1}^n (\sigma \sigma^T)_{i,j}(t, x) p_j, i = 1, \dots, d$$

and

$$\gamma(t, x, u, p) = -\langle b(t, x), p \rangle + \sum_{i,j=1}^d \frac{\partial}{\partial x_i} (\sigma \sigma^T)_{i,j}(t, x) p_j - c(t, x) u.$$

Assume that there are positive constants $\nu, \mu > 0$ such that for every $\xi \in \mathbb{R}^d$ the matrix $\left[(\sigma \sigma^T)_{i,j}(t, x) \right]_{i,j=1}^d$ satisfies

$$\nu |\xi|^2 \leq \sum_{i,j=1}^d (\sigma \sigma^T)_{i,j}(t, x) \xi_i \xi_j \leq \mu |\xi|^2$$

and that the coefficients b and c are such that

$$\left\| \sum_{i=1}^d b_i^2 \right\|_{q,r,\Omega_T} + \|c\|_{q,r,\Omega_T} \leq \mu, \text{ for some } \mu > 0$$

where we recall for example $\|c\|_{q,r,\Omega_T} = \left(\int_0^T \left(\int_{\Omega} |c(t, x)|^q dx \right)^{r/q} dt \right)^{1/r}$ and r, q satisfy the relations

$$\begin{aligned} \frac{1}{r} + \frac{d}{2q} &= 1 \\ q &\in (d/2, \infty], r \in [1, \infty), \text{ for } d \geq 2, \\ q &\in [1, \infty], r \in [1, 2], \text{ for } d = 1. \end{aligned}$$

In particular, the previous bounds always hold in the case of coefficients b and c that are bounded in Ω_T .

Under these conditions, standard a-priori bounds for linear PDE's, see Chapter III of [27], show that Theorem 5.4 holds. In addition, in the linear case an even stronger result holds. Theorem 4.5 of Chapter III of [27] implies that f^n converges strongly to u , the unique weak solution to (5.1), in $W^{1,2}(\Omega_T)$.

6 Conclusion

We believe that deep learning could become a valuable approach for solving high-dimensional PDEs. The PDE solution can be approximated with a deep neural network which is trained to satisfy the differential operator, initial condition, and boundary conditions. We prove that the neural network converges to the solution of the partial differential equation as the number of hidden units increases.

Our deep learning algorithm for solving PDEs is meshfree, which is key since meshes become infeasible in higher dimensions. Instead of forming a mesh, the neural network is trained on batches of randomly sampled time and space points. The approach is implemented for American options (a type of free-boundary PDE which is widely used in finance) in up to 200 dimensions with accurate results.

It is also important to put the numerical results in Section 4 in a proper context. PDEs appearing in finance (such as the American option free-boundary PDE) typically have monotonic solutions. PDEs with non-monotonic or highly oscillatory solutions are more challenging to solve and further developments in architecture may be necessary. Further numerical development and testing is therefore required to better judge the usefulness of deep learning for the solution of PDEs in other applications.

References

- [1] J. Sirignano and K. Spiliopoulos, Stochastic Gradient Descent in Continuous Time, 2017, forthcoming in *SIAM Journal on Financial Mathematics*.
- [2] Justin Sirignano, Deep Learning Models in Finance, *SIAM News*, 2017.
- [3] J. Tompson, K. Schlachter, P. Sprechmann, and K. Perlin. Accelerating Eulerian Fluid Simulation with Convolutional Networks, *Proceedings of Machine Learning Research*, Vol. 70, pgs. 3424-3433, 2017.
- [4] P. Chaudhari, A. Oberman, S. Osher, S. Soatto, and G. Carlier. Deep relaxation: partial differential equations for optimizing deep neural networks, 2017.
- [5] I. Lagaris, A. Likas, and D. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Transactions on Neural Networks*, Vol. 9, No. 5, pgs. 987-1000, 1998.

- [6] I. Lagaris, A. Likas, and D. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, *IEEE Transactions on Neural Networks*, Vol. 11, No. 5, pgs. 1041-1049, 2000.
- [7] K. Rudd, Solving Partial Differential Equations using Artificial Neural Networks, *PhD Thesis*, Duke University, 2013.
- [8] H. Lee, Neural Algorithm for Solving Differential Equations, *Journal of Computational Physics*, Vol. 91, pgs. 110-131, 1990.
- [9] A. Malek and R. Beidokhti, Numerical solution for high order differential equations using a hybrid neural network-optimization method, *Applied Mathematics and Computation*, Vol. 183, No. 1, pgs. 260-271, 2006.
- [10] W. E., J. Han, and A. Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations, *Communications in Mathematics and Statistics*, Springer, 2017.
- [11] C. Beck, W. E., A. Jentzen. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations, [arXiv:1709.05963](https://arxiv.org/abs/1709.05963) 2017.
- [12] M. Fujii, A. Takahashi, and M. Takahashi. Asymptotic Expansion as Prior Knowledge in Deep Learning Method for high dimensional BSDEs, [arXiv:1710.07030](https://arxiv.org/abs/1710.07030), 2017.
- [13] F. Longstaff and E. Schwartz, Valuing American Options by Simulation: A Simple Least-Squares Approach, *Review of Financial Studies*, Vol. 14, pgs. 113-147, 2001.
- [14] L.C.G. Rogers, Monte-Carlo Valuation of American Options, *Mathematical Finance*, Vol. 12, No. 3, pgs. 271-286, 2002.
- [15] M. Haugh and L. Kogan, Pricing American Options: A Duality Approach, *Operations Research*, Vol. 52, No. 2, pgs. 258-270, 2004.
- [16] C. Reisinger and G. Wittum, Efficient hierarchical approximation of high-dimensional option pricing problems, *SIAM Journal on Scientific Computing*, Vol. 29, No. 1, pgs. 440-458, 2007.
- [17] C. Reisinger, Analysis of linear difference schemes in the sparse grid combination technique, *IMA Journal of Numerical Analysis*, Vol. 33, No. 2, pgs. 544-581, 2012.
- [18] A. Heinecke, S. Schraufstetter, and H. Bungartz, A highly parallel Black-Scholes solver based on adaptive sparse grids, *International Journal of Computer Mathematics*, Vol. 89, No. 9, pgs. 1212-1238, 2012.
- [19] H. Bungartz, A. Heinecke, D. Pfluger, and S. Schraufstetter, Option pricing with a direct adaptive sparse grid approach, *Journal of Computational and Applied Mathematics*, Vol. 236, No. 15, pgs. 3741-3750, 2012.
- [20] H. Bungartz and M. Griebel, Sparse Grids, *Acta numerica*, Vol. 13, pgs. 174-269, 2004.
- [21] D. Bertsekas and J. Tsitsiklis, Gradient convergence in gradient methods via errors, *SIAM Journal of Optimization*, Vol.10, No. 3, pgs. 627-642, 2000.
- [22] L. Boccardo, A. Dall'Aglio, T. Gallouët and L. Orsina, Nonlinear parabolic equations with measure data, *Journal of Functional Analysis*, Vol. 147, pp. 237-258, (1997).
- [23] L. Boccardo, M.M. Porzio and A. Primo, Summability and existence results for nonlinear parabolic equations, *Nonlinear Analysis: Theory, Methods and Applications*, Vol. 71, Issues 304, pp. 1-15, 2009.
- [24] G. Cybenko, Approximation by superposition of a sigmoidal function, *Mathematics of Control, Signals and Systems*, Vol. 2, pp. 303-314, 1989.

- [25] K. Hornik, M. Stinchcombe and H. White, Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks, *Neural Networks*, Vol. 3, Issue 5, pp. 551-560, 1990.
- [26] K. Hornik, Approximation capabilities of multilayer feedforward networks, *Neural Networks*, Vol. 4, pp. 251-257, 1991.
- [27] O. A. Ladyzenskaja, V. A. Solonnikov and N. N. Ural'ceva, Linear and Quasi-linear Equations of Parabolic Type (Translations of Mathematical Monographs Reprint), *American Mathematical Society*, Vol. 23, 1988.
- [28] M. Magliocca, Existence results for a Cauchy-Dirichlet parabolic problem with a repulsive gradient term, *Nonlinear Analysis*, Vol. 166, pp. 102-143, 2018
- [29] R. Di Nardo, F. Feo, O. Guibé, Existence result for nonlinear parabolic equations with lower order terms, *Anal. Appl.(Singap.)*, Vol. 09, No. 02, pp. 161-186, 2011.
- [30] M. M. Porzio, Existence of solutions for some “noncoercive” parabolic equations, *Discrete and Continuous Dynamical Systems*, Vol. 5, Issue 3, pp. 553-568, 1999.
- [31] J. Simon, Compact sets in the space $L^p(0, T; B)$, *Annali di Matematica Pura ed Applicata*, Vol. 146, pp. 65-96, 1987.